Group Project

Database Design of Food Order Platform

Course: Database Management and Applications (1001)

Tutor: Dr. Lily Bei LUO

Member

Ziheng LYU 2330036105
Zeming CHEN 2330036010
Jiaming LIU 2330036089
Xin WU 2330036159
Ang MA 2330036106

Contents

1 Planning **2**

1.1 Enterprise modeling . . . 2
1.2 Conceptual data modeling . . . 2
1.3 Further analysis plan . . . 3

2 Analysis **3**

2.1 Commercial Activity . . . 3
2.2 E-R Constructs . . . 4
2.2.1 Purpose . . . 4
2.2.2 Entities . . . 4
2.2.3 Attributes . . . 5
2.2.4 Relationships . . . 7
2.2.5 Assosiative Entity . . . 7
2.3 EER Constructs . . . 8
2.3.1 Generalization to SELFPICK and DELIVERY . . . 8
2.3.2 Constraints in Super-type and Sub-type Relationships . . . 8
2.3.3 Supertype and Subtype Relationships Constrains: . . . 8
2.4 ERD . . . 8
2.5 Logical Schema . . . 10
2.5.1 Normal Forms . . . 10
2.5.2 Mapping rules . . . 10

3 Data Analysis **13**

3.1 Analysis of ORDER . . . 13
3.2 Analysis for ORDERLINE . . . 13
3.3 Analysis for CUSTOMER . . . 14
3.4 Analysis on DELIVERY . . . 15

4 Database sustainable development suggestions **15**

4.1 Implementation . . . 15
4.1.1 Problem . . . 15
4.1.2 Suggestion . . . 16
4.2 Maintenance . . . 16
4.2.1 Problem . . . 16
4.2.2 Suggestions . . . 16

5 Appendix **17**

1 Planning

In this section, the object is to develop a database structure based on the food order industry and generally introduce the information system to help solve business problems.

According to the data, for the frequency of Chinese food order platform consumers ordering food in 2023, 4.08% of consumers ordering food online more than 20 times a week, 13.52% of consumers ordering 11-20 times a week, 32.78% of consumers ordering 5-10 times a week. 23.98% of consumers order takeaway food 3-4 times a week, and 16.45% of consumers order takeaway food 1-2 times a week, demonstrating that Chinese citizens have strong demands to order food online (Wu, 2024). The trend in the fast food industry is irreversible. Therefore, we design a food order platform database to expand the merchant's network business and improve the timeliness of ordering services for users.

1.1 Enterprise modeling

The core of this food order application system is to manage multiple orders and operate the delivery process which are basic elements for online food service.

For the traditional food order system, due to the online techniques deficiency, the relationship is limited, insufficiently involving around merchants and customers. This original database can simply deal with the food order problems offline, lacking flexibility and diversity of food order means. For the new database project, we expand the supply chain, applying online order platforms and delivery services to better suit the rapid development of food service industry. Therefore, the relationship will be extended to the processing cycle of merchant, delivery, order, and customer. And customers can choose whether to take out food or use a delivery service The data types include string, float, datetime, etc. The function of this database involves online food order applications, merchant supply management, and delivery operations.

1.2 Conceptual data modeling

For the conceptual data modeling, the initiative Entity-Relation-Diagram will be provided to logically demonstrate the running business process of food order system.

The requirements refer to systematically elaborating the food order platform database and improving the efficiency and creditability of the database. For instance, the database can store information on sale(merchant), customer, order, delivery and deliverer, adding connection by orderlines, providing multiple choices for what food to choose, how to order food, how to deliver food, how to evaluating merchants' performance, how to justify delivers' performance, how to comment on food.

1.3 Further analysis plan

ERD Transform Entity-Relationship-Diagram to the Enhance ER model, adding subtypes in Order, separating delivery into takeout and selfpick, connecting delievery relationship with takeout order.

Logical Schema Design a logical schema based on the enhanced ER model, remove data redundancy (covert logical schema to 3 normal form), show functional dependency, provide detailed review of the business transaction.

RDBMS Utilize MySQL(Relational Database Management System) to do business experiment, creating database, creating tables for entities, inserting different types of attributes, adding values, analyzing the marketing situation of different merchants and evaluating delivery performance.

2 Analysis

2.1 Commercial Activity

1. Food Ordered by Customers
    - Customers browse the available food options on the platform.
    - They select food items and place an order.
    - Each order is associated with specific food items and quantities.
2. Order Type Selection by Customers
    - Customers choose between pick-up or delivery when placing an order.
3. Delivery Process
    - Once an order is placed, the system assigns a deliverer to transport the food from seller to customer.
    - The deliverer retrieves the ordered food items from the seller's location.
    - They deliver the food to the customer's address within a period of time.
4. Order Created by Customers
    - Customers create orders by selecting desired food, and providing payment.
5. Customer Ratings for Merchants
    - After receiving their orders, customers have the opportunity to rate and review the merchants based on their overall experience.

6. Customer Ratings for Deliverers:

    - Upon receiving the delivery, customers can rate for the deliverer's performance.
    - Ratings may consider factors such as delivery speed, professionalism, and order accuracy.

2.2 E-R Constructs

2.2.1 Purpose

- Clearly Describe the Business Logic of Online Food Order System: ER diagram visualizes participants (sellers, customers, deliverer) and information of entities (attributes) needed to demonstrate on the system. It also shows the relationships and interactions among entities, facilitating an understanding of its fundamental working order for users.
- Provide a Foundation for Database Design: ER diagram acts as groundwork for database design. We can determine the Enhanced E-R diagram, table structure and relationships of the database according to the E-R diagram.

2.2.2 Entities

Based on the background of food delivery service industry, we are going to design a food delivery data platform. Hence, five basic entities can be designed in our Entity-Relationship diagram, they are SALES, FOOD, ORDER, CUSTOMER, and DELIVERER.

SALES A sales entity represents merchants who have registered on the delivery platform. A sales entity is uniquely identified by a SalesID. It excludes merchants who have applied for but not yet been approved for inclusion. A sales instance is created when a merchant successfully completes registration and becomes operational on the platform.

FOOD A food entity is a product available for purchase, typically consisting of dishes offered by a seller. A food entity is identified by a FoodID. It does not include product bought through other platforms. A food instance is implicitly created when the seller provides it for sale or when it becomes available for purchase in the system.

CUSTOMER A customer is a user who place orders for food on the platform. A customer is identified by a CustomerID. Customers do not include users who purchase food through other platforms. A customer instance is implicitly established when a person completes registration and becomes a user on the platform.

DELIVERER A deliverer is an individual responsible for transporting food items from the seller to the customer. A deliverer is identified by a unique DelivererID. Deliverers only include in the delivery process within the platform and are not associated with deliveries

made through external platforms. A deliverer instance is established when an individual completes platform registration.

ORDER An ORDER is a record of customers placing orders for food through the platform. Each order is uniquely identified by an OrderID. It doesn't include purchases made through alternative platforms. An order instance is generated when a customer initiates a purchase via the platform.

2.2.3 Attributes

SALES **SalesID** A unique number assigned to a seller based on the order in which seller entered the platform. It is numeric as it serves as a unique identifier. It is required for every seller. Do not allow updates. It is a identifier and should not change once assigned to a seller.

SalesName The name of the seller as displayed on the platform. String. It is required for every seller. Allow updates. The seller's name might change.

SalesAddress The physical address of the seller's store. String. It is required for every seller in order to make sure customers know where their food produced and delivered. Allow updates. The seller's address might change.

SalesRating An average numerical score reflecting customer reviews and feedback for the seller. It is a Float type. It is optional. Because new sellers who initial setup do not have score. Allow updates. It is expected to change over time based on new reviews.

SalesPhoneNo The phone number associated with the seller for customer contact. It is a String type. It is required for every seller. Hence, customers can contact them when they in need. Allow updates. The seller's phone number might change.

SalesCount The total number of orders placed from this seller on the platform. It is an Integer type. It is required. Provide data to customers which is easy for them to compare. Allow updates. It is expected to increase as customers place orders from the seller.

FOOD **FoodID** A unique identifier for each food item offered on the platform. It is numeric as it serves as a unique identifier. It is required for every food item. Do not allow updates. Because it is an identifier and should not change once assigned to a food item.

SalesID The identifier of the seller associated with this food item. Integer. It is required. Because every food item must be associated with its seller. Allow updates. In some cases, a food item might remove or add from the menu.

FoodName The name of the food item as displayed on the platform. String. It is required for every food item. Allow updates. Because the food item's name might change.

FoodPrice The price of the food item in the platform's base currency. Float. It is required for every food item. Allow updates. Because it might change due to promotions, cost adjustments, or other factors.

FoodRating An average numerical score reflecting customer reviews and feedback for the food item. Float. It is optional. Because new food item which initial setup do not have score. Allow updates. It is expected to change over time based on new reviews.

CUSTOMER **CustomerID** A unique identifier for each customer registered on the platform. It is numeric as it serves as a unique identifier. It is required for every customer. Do not allow updates. Because it is an identifier and should not change once assigned to a customer.

CustomerName The name of the customer as displayed on the platform. String. It is required for every customer Allow updates. Because customer's name might change.

CustomerAddress The physical address of the primary residence or delivery location. String. It is optional for customer. Due to some customers have not ordered take-out after registering, they do not need to save their address. Allow updates. The customer's address might change.

CustomerPhoneNo The phone number associated with the customer for communicate. String. It is required for every customer. Hence, deliverers can contact them when they need it. Allow updates. The customer's phone number might change.

CustomerCount The total number of orders placed from customer on the platform. Integer. It is required. Record data of customers and make it comfortable for database users to analyze the data. Allow updates. It is expected to increase as customers place orders from the sellers.

DELIVERER **DelivererID** A unique identifier for each deliverer registered on the platform. It is numeric as it serves as a unique identifier. It is required for every deliverer. Do not allow updates. Because it is an identifier and should not change once assigned to a deliverer.

DlivererName The name of the deliverer as displayed on the platform. String. It is required for every deliverer Allow updates. Because the deliverer's name might change.

DelivererRating An average numerical score reflecting customer reviews and feedback for the deliverer Float. It is optional. Because new deliverers who initial setup do

not have score. Allow updates. It is expected to change over time based on new reviews.

DelivererCount The total number of orders completed by the deliverer on the platform. Integer. It is required. Show real data to customer and make it comfortable for database users to analyze the data. Allow updates. It is expected to increase as deliverers complete the order.

ORDER **OrderID** A unique identifier for each order placed on the platform. It is numeric as it serves as a unique identifier. It is required for every order. Do not allow updates. Because it is an identifier and should not change as once assigned to an order.

OrderTime The timestamp when the order was placed. Timestamp. It is required for every order. Do not allow updates. OrderTime should not change once the order is placed.

OrderPrice The total price of the order, including the prices of all ordered food. Float. It is required for every order. Do not allow updates. It is calculation results.

OrderType The type of order, such as delivery and self-pick. String. It is required for every order. Allow updates. An order type might need to be changed (e.g., from delivery to self-pick).

2.2.4 Relationships

SALES and FOOD one-to-many relationship. Suppose a merchant must provide at least one item, and can have many and a dish is provided by one and only merchant.

FOOD and ORDER many-to-many relationship. Suppose a dish can be contained in many orders or may not be contained to any at all, and an order must contain at least one dish, and can have many.

ORDER and CUSTOMER one-to-many relationship. Suppose an order must be created by one and only one customer, and a customer can create many orders or may not create any at all. In addition, suppose an order must be received by one and only one customer, and a customer can receive many orders or may not receive any at all.

2.2.5 Assosiative Entity

Since the relationship between FOOD and ORDER is many-to-many, it should be replaced by two one-to-many relationships with the associative entity called ORDERLINE. And Quantity is its attribute.

2.3 EER Constructs

2.3.1 Generalization to SELFPICK and DELIVERY

SELFPICK and DELIVERY have common attributes of ORDER.

SELFPICK has unique attribute that are ExpectedPickTime and ActualPickTime.

DELIVERY has unique attributes that are StartDeliveryTime, EndDeliveryTime, DeliverRating, and DeliveryDistance.

2.3.2 Constraints in Super-type and Sub-type Relationships

Total Specialization An ORDER has to be either SELFPICK or DELIVERY.

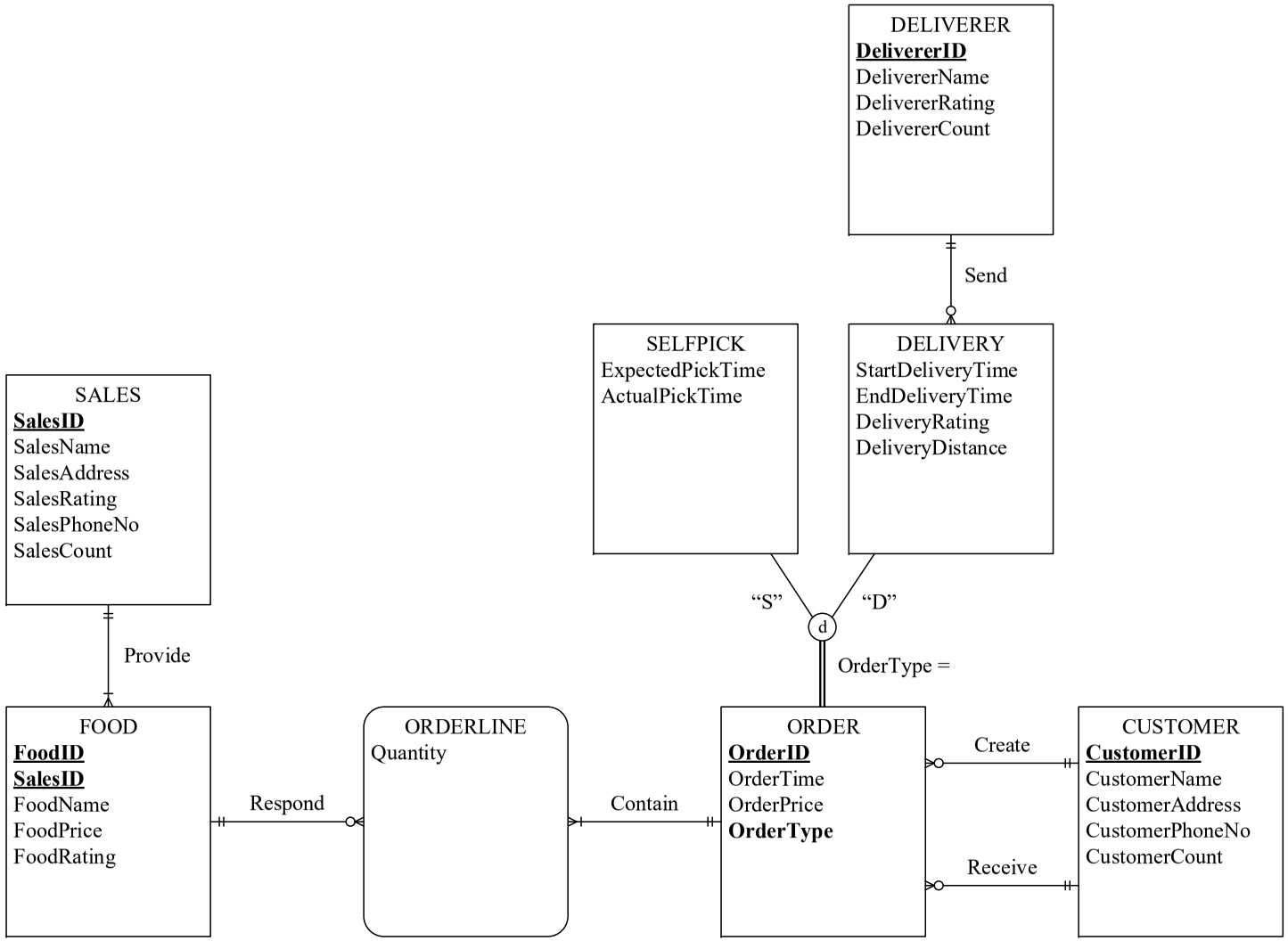
Disjoint rule An ORDER can be either SELFPICK or DELIVERY, but not both at the same time.

2.3.3 Supertype and Subtype Relationships Constrains:

DELIVERY and DELIVERER one-to-many relationship. Suppose a takeaway order must be delivered by one and only one deliverer, and a deliverer can deliver many orders or may not deliver any at all.

2.4 ERD

The ERD is shown below.


DELIVERER
DelivererID
DelivererName
DelivererRating
DelivererCount
Send
SELFPICK
ExpectedPickTime
ActualPickTime
DELIVERY
StartDeliveryTime
EndDeliveryTime
DeliveryRating
DeliveryDistance
SALES
SalesID
SalesName
SalesAddress
SalesRating
SalesPhoneNo
SalesCount
"S"
"D"
d
Provide
OrderType =
FOOD
FoodID
SalesID
FoodName
FoodPrice
FoodRating
Respond
ORDERLINE
Quantity
Contain
ORDER
OrderID
OrderTime
OrderPrice
OrderType
Create
Receive
CUSTOMER
CustomerID
CustomerName
CustomerAddress
CustomerPhoneNo
CustomerCount

2.5 Logical Schema

A relation is a named, two-dimensional table of data. A logical schema, in the context of databases, refers to the design or blueprint of how data is organized and represented in a database system. It defines the structure of the database without specifying the physical implementation details like storage mechanisms or indexing methods, focusing instead on the logical relationships between data elements. After we obtain the ERD, we need to convert it to a Logical Schema based on the ERD.

2.5.1 Normal Forms

Logical Schema is divided into three Normal Forms: 1st NF, 2nd NF, and 3rd NF. Among them, the 1st NF is a direct conversion, the 2nd NF is to remove the partial functional dependency, and the 3rd NF is to delete the transitive dependency.

2.5.2 Mapping rules

For Transforming EER Diagrams into Relations, we should map regular entities to relations by following such rules:

Simple attributes E-R attributes map directly onto the relation

Composite attributes Use only their simple, component attributes

Multi-valued attributes Become a separate relation with a foreign key taken from the superior entity

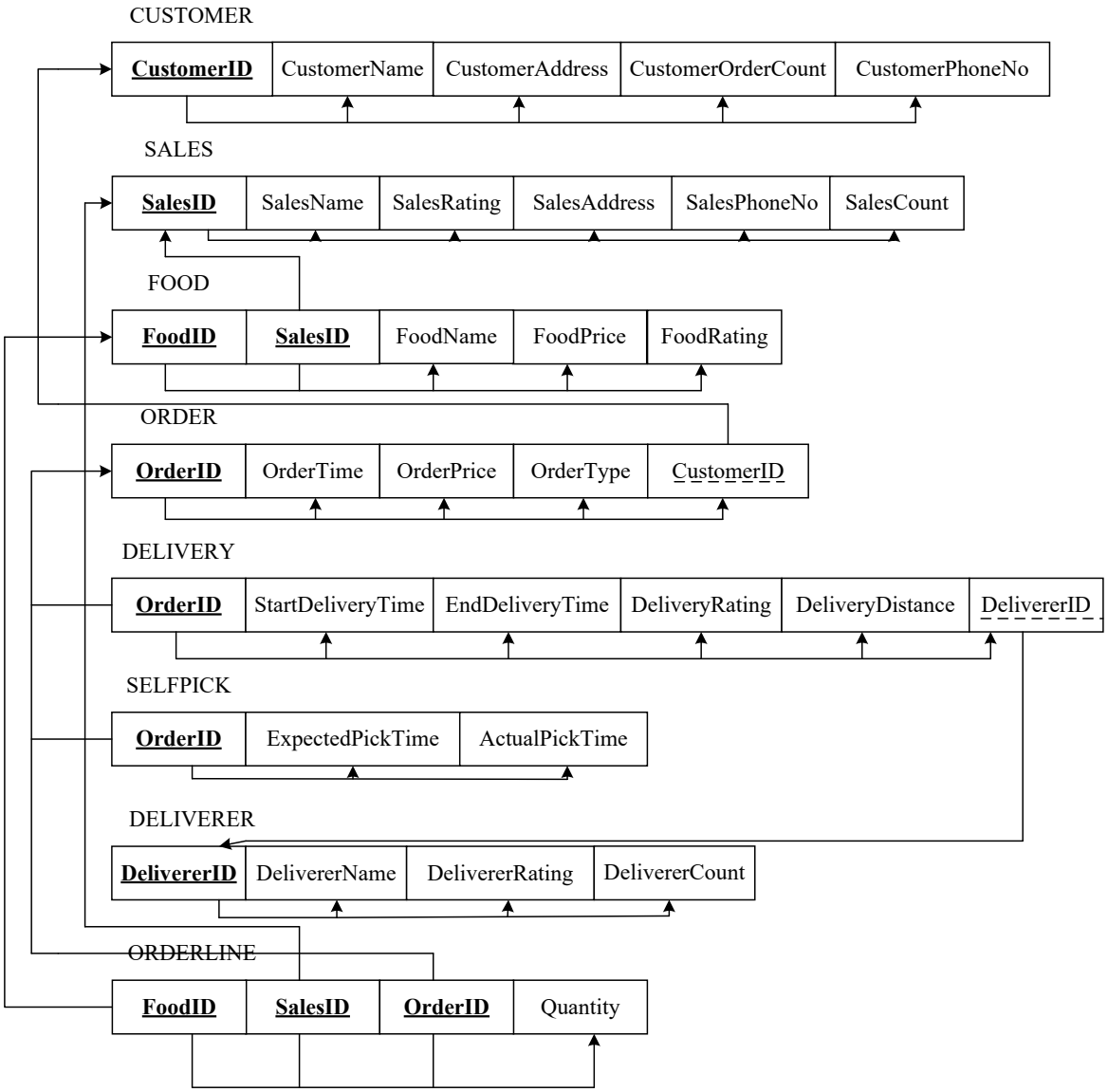
Firstly, we need to transform EER Diagrams into Relations. The name given to the relation is the same as the entity type. Each simple attribute of the entity type becomes an attribute of the relation. The identifier of the entity type becomes the primary key of the relations. Because all the relationship in the ERD are binary relationships, we then just focus on map binary relationships and super-type/sub-type:

One-to-Many Create a relation for each of the two entity types. Primary key on the one side becomes a foreign key on the many side.

Many-to-Many Create relations for entities and create a new relation for associate entity. If associate entity does not have its identifier, the primary key of each of the two participating entities become the primary key of associate entity.

Super-type/Sub-type Create a separate relation for the super-type and for each of its subtypes. Assign the attributes of super-type to its relation. Assign to the relation for each sub-type the primary key of the super-type and only those attributes that are unique to that sub-type.

By designing that reference, the schema consistency and accuracy can be confirmed. Based on the referential integrity and the functional dependency, this database can be reliably applied to complete customer ordering service, merchants receiving orders, platform delivery services, user evaluation, and tracking processes according to the record time. We can conduct the Logical Schema as shown below.


CUSTOMER
CustomerID
CustomerName
CustomerAddress
CustomerOrderCount
CustomerPhoneNo
SALES
SalesID
SalesName
SalesRating
SalesAddress
SalesPhoneNo
SalesCount
FOOD
FoodID
SalesID
FoodName
FoodPrice
FoodRating
ORDER
OrderID
OrderTime
OrderPrice
OrderType
CustomerID
DELIVERY
OrderID
StartDeliveryTime
EndDeliveryTime
DeliveryRating
DeliveryDistance
DelivererID
SELFPICK
OrderID
ExpectedPickTime
ActualPickTime
DELIVERER
DelivererID
DelivererName
DelivererRating
DelivererCount
ORDERLINE
FoodID
SalesID
OrderID
Quantity

3 Data Analysis

The data generated during use holds vital information, the information, however, is not the mere collection of numbers, but rather what the numbers represent from there we may gain crucial insights that aid in understanding user behaviors, market trends, and business performance. The information came from the analysis of the various data can guide us in formulating more effective marketing strategies, optimizing operational processes, improving products and services, and enhancing user experiences.

3.1 Analysis of ORDER

In order to find out the total number of orders placed within a specific time period (day, week, month), The code developed for this intention:

```
SELECT COUNT(orderid) AS total_orders
FROM orders
WHERE orderdate BETWEEN 'start_date' AND 'end_date';
```

We categorized orders based on types: takeaway and pick-up. To determine the percentage distribution of each order type, the code developed for this intention:

```
SELECT ordertype, COUNT(*) * 100 /
(SELECT COUNT(*) FROM orders) AS percentage
FROM orders
GROUP BY ordertype;
```

3.2 Analysis for ORDERLINE

To find out the average order amount, the code we designed for this purpose:

```
SELECT AVG(order_amount) AS average_order_amount
FROM orders;
```

To learn the median order amount, we designed:

```
SELECT
  AVG(order_amount) AS median_order_amount
FROM (
  SELECT order_amount
  FROM orders
  ORDER BY order_amount
  LIMIT 1 OFFSET (SELECT COUNT(*) FROM orders) / 2
) AS median_subquery;
```

To determine the mode of order amount, this is the code we designed for this purpose:

```
SELECT order_amount, COUNT(order_amount) AS frequency
FROM orders
GROUP BY order_amount
ORDER BY frequency DESC
LIMIT 1;
```

To find the maximum and minimum values of order amount, the code we designed for this purpose:

```
SELECT MAX(order_amount) AS max_order_amount,
MIN(order_amount) AS min_order_amount
FROM orders;
```

To find out the range of order amounts, the code we designed for this purpose:

```
SELECT MAX(order_amount) - MIN(order_amount)
AS order_amount_range
FROM orders;
```

To discover the relationship between order amounts and other factors such as order type, time of day, or menu items ordered, this is the code we designed:

```
SELECT ordertype, AVG(order_amount) AS avg_order_amount
FROM orders
GROUP BY ordertype;
```

For determine the top-selling menu items, we constructed this code:

```
SELECT FoodID, COUNT (*) AS total_orders
FROM orders GROUP BY FoodID
ORDER BY total_orders DESC LIMIT 5;
```

3.3 Analysis for CUSTOMER

In order to identify popular Foods, seasonal trends, and changes in customer preferences over time, this is the code we designed:

```
SELECT foodid, COUNT(*) AS total_orders
FROM orders
GROUP BY foodid
ORDER BY total_orders DESC
LIMIT 5;
```

In order to identify the average frequency at which customers place orders (daily, weekly, monthly), the code tailored for this objective:

```
SELECT customerid, AVG(order_frequency)
AS avg_order_frequency
FROM (
  SELECT customerid, COUNT(order_id) AS order_frequency
  FROM orders
  GROUP BY customerid
) AS order_freq_subquery
GROUP BY customerid;
```

3.4 Analysis on DELIVERY

To examine the average time taken to fulfill orders from placement to delivery, we programmed this code:

```
SELECT AVG(TIMESTAMPDIFF(MINUTE, startdeliverytime, enddeliverytime))
AS avg_delivery_time
FROM delivery;
```

4 Database sustainable development suggestions

Based on systems development life cycle (SDLC), we have completed the first three steps which are Planning, Analysis, Design. Nevertheless, Implementation and Maintenance as the essential parts in SDLC are missing. In this way, the Food-order platform exhibits tendencies toward instability. Therefore, it is of importance to demonstrate the problems may face and provide suggestions to handle the issues. The following is the detailed illustration.

4.1 Implementation

Implementation involves coding, testing and deployment which ensures software compliance with design specifications and functional integrity. It should be noted, we have already finished the coding part by right of using MySQL. Meanwhile, we have done few tests to tackle some syntax or logical errors.

4.1.1 Problem

Nevertheless, most of testing and whole deployment phase are not be processed, which indicates that our designed project is deficient in practical experience and may result in a negative situation where the project may fail to verify the assumptions and decisions during the design

and analysis stages. Simultaneously, without deployment may lead to a case where we could not evaluate performance and fitness of our Food-Order platform.

4.1.2 Suggestion

In order to furnish workable suggestions, we avoid utilizing actual platform deployment and replacing by using MATLAB or Python to simulate and analyze vital function and performance. For example, MATLAB can simulate the response of data handling modules under high data inputs rate. By virtue of this method, students can make a quantitative evaluation with no actual users.

4.2 Maintenance

Maintenance phase which encompasses fixing defects, updating system and adapting new demanding or changes in circumstances provides ongoing monitor, support and enhancement of the system.

4.2.1 Problem

Typically, lack of the capacity and resources for maintenance tasks is a thorny issue student confront. Although few students could do a small quantity of maintenance work, long-term maintenance can not be applied. In addition, without professional and effective way to maintain our platform, the platform may rapidly become outdated or uncorrelated due to lingering errors or obsolete designs.

4.2.2 Suggestions

We conclude suggestions from two perspectives:

(1) Considering opening the project in online community is a suitable way to obtain auxiliary maintenance. This solution can not only enable others to prompt the project directly under collaboration, but also attract developers and users who may access and utilize to provide feedback to resolve issues.

(2) Establishing a project inheritance plan to transfer the project to incoming students can be a useful way. This passing on the baton of project promotes project in a sustainable development environment by constant improvement from generations of students.

(3) With a view to assist platform and sellers to make adjustments, we can also visualize by using SQL to draw histograms or box plots to identify the issues and tackle them in time.

In conclusion, the most crucial thing is maximizing the academic and practical value of the project. We believe students can have their comprehensive understanding of the entire project from the hands-on experience if they invest adequate energy and concentrate on the project, though the project does not undergo a full systems development life cycle.

5 Appendix

- Create Database.

```
CREATE DATABASE GroupProject;
USE GroupProject;
```

- Create CUSTOMER Table.

```
CREATE TABLE CUSTOMER(
CustomerID int NOT NULL,
CustomerName VARCHAR(30) NOT NULL,
CustomerAddress VARCHAR(100),
CustomerOrderCount INT NOT NULL,
CustomerPhoneNo VARCHAR(20) NOT NULL,

CONSTRAINT CUSTOMER_PK PRIMARY KEY (CustomerID));
```

- Create SALES Table.

```
CREATE TABLE SALES(
SalesID int NOT NULL,
SalesName VARCHAR(30) NOT NULL,
SalesRating DECIMAL(2),
SalesAddress VARCHAR(100),
SalesPhoneNo VARCHAR(20) NOT NULL,
SalesCount INT,

CONSTRAINT SALES_PK PRIMARY KEY (SalesID));
```

- Create DELIVERER Table.

```
CREATE TABLE DELIVERER(
DelivererID int NOT NULL,
DelivererName VARCHAR(30) NOT NULL,
DelivererRating DECIMAL(2),
DelivererCount INT,

CONSTRAINT DELIVERER_PK PRIMARY KEY (DelivererID));
```

- Create FOOD Table.

```
CREATE TABLE FOOD(
FoodID int NOT NULL,
SalesID int NOT NULL,
FoodName VARCHAR(30) NOT NULL,
FoodPrice DECIMAL(2) NOT NULL,
FoodRating DECIMAL(2),

CONSTRAINT FOOD_PK1 PRIMARY KEY (FoodID),
CONSTRAINT FOOD_PK2 PRIMARY KEY (SalesID));
```

- Create ORDER Table.

```
CREATE TABLE ORDER(
OrderID int NOT NULL,
OrderTime DATETIME NOT NULL,
OrderType VARCHAR(2) NOT NULL,
CustomerID int NOT NULL,

CONSTRAINT ORDER_PK PRIMARY KEY (OrderID)
CONSTRAINT ORDER_FK1 FOREIGN KEY (CustomerID)
                            REFERENCES CUSTOMER(CustomerID));
```

- Create SELFPICK Table.

```
CREATE TABLE SELFPICK(
OrderID int NOT NULL,
ExpectedPickTime DATETIME NOT NULL,
ActualPickTime DATETIME NOT NULL,

CONSTAINT SELFPICK_PK PRIMARY KEY (OrderID));
```

- Create DELIVERY Table

```
CREATE TABLE DELIVERY(
OrderID int NOT NULL,
StartDeliveryTime DATETIME NOT NULL,
EndDeliveryTime DATETIME NOT NULL,
DeliveryRating DECIMAL(2) NOT NULL,
DeliveryDistance DECIMAL(2) NOT NULL,

CONSTAINT DELIVERY_PK PRIMARY KEY (OrderID));
```

- Create ORDERLINE Table.

```
CREATE TABLE ORDERLINE(
FoodID int NOT NULL,
SalesID int NOT NULL,
OrderID int NOT NULL,
Quantity INT NOT NULL,

CONSTRAINT ORDERLINE_PK1 PRIMARY KEY (FoodID),
CONSTRAINT ORDERLINE_PK2 PRIMARY KEY (SalesID),
CONSTRAINT ORDERLINE_PK3 PRIMARY KEY (OrderID));
```

- Insert values.

```
INSERT INTO Customer
(CustomerID, CustomerName, CustomerAddress, CustomerPhoneNo, CustomerCount)
VALUES
(123457, 'Jane Smith', '456 Elm St, Somewhereville USA', '(456) 789-0123', 5),
(123458, 'Mike Johnson', '789 Oak St, Nowhereville USA', '(789) 012-3456', 8),
(123459, 'Emily Brown', '321 Pine St, Anyplace USA', '(012) 345-6789', 15),
(123460, 'David Wilson', '654 Maple St, Elsewhere USA', '(234) 567-8901', 7),
(123461, 'Sarah Garcia', '987 Birch St, Noway USA', '(345) 678-9012', 12),
(123462, 'Chris Martinez', '852 Cedar St, Anywhere USA', '(456) 789-0123', 9),
(123463, 'Jessica Rodriguez', '741 Walnut St, Elsewhereville USA', '(567) 890-123
(123464, 'Daniel Hernandez', '369 Spruce St, Nowhere USA', '(678) 901-2345', 11)
(123465, 'Amanda Lopez', '258 Birch St, Anyplaceville USA', '(789) 012-3456', 14)
(123466, 'Michael Gonzalez', '147 Cedar St, Somewhere USA', '(890) 123-4567', 8)
(123467, 'Ashley Perez', '963 Pine St, Nowayville USA', '(901) 234-5678', 10),
(123468, 'Matthew Sanchez', '852 Oak St, Elsewhereland USA', '(012) 345-6789', 13
(123469, 'Jennifer Ramirez', '741 Elm St, Anytown USA', '(123) 456-7890', 9),
(123470, 'Christopher Torres', '369 Maple St, Elsewhereville USA', '(234) 567-890
(123471, 'Jessica Flores', '258 Pine St, Anywhereville USA', '(345) 678-9012', 7)
(123472, 'Joshua Gomez', '963 Cedar St, Nowayland USA', '(456) 789-0123', 11),
(123473, 'Amanda Brooks', '852 Spruce St, Somewhereville USA', '(567) 890-1234',
(123474, 'James Bell', '741 Birch St, Anyplace USA', '(678) 901-2345', 13),
(123475, 'Danielle Ward', '369 Pine St, Nowhere USA', '(789) 012-3456', 6),
(123476, 'Andrew Cox', '258 Oak St, Elsewhere USA', '(890) 123-4567', 9);
(123457, 'Jane Smith', '456 Elm St, Somewhereville USA', '(456) 789-0123', 5),
(123458, 'Mike Johnson', '789 Oak St, Nowhereville USA', '(789) 012-3456', 8),
(123459, 'Emily Brown', '321 Pine St, Anyplace USA', '(012) 345-6789', 15),
```

```
(123460, 'David Wilson', '654 Maple St, Elsewhere USA', '(234) 567-8901', 7),
(123461, 'Sarah Garcia', '987 Birch St, Noway USA', '(345) 678-9012', 12),
(123462, 'Chris Martinez', '852 Cedar St, Anywhere USA', '(456) 789-0123', 9),
(123463, 'Jessica Rodriguez', '741 Walnut St, Elsewhereville USA', '(567) 890-123
(123464, 'Daniel Hernandez', '369 Spruce St, Nowhere USA', '(678) 901-2345', 11)
(123465, 'Amanda Lopez', '258 Birch St, Anyplaceville USA', '(789) 012-3456', 14
(123466, 'Michael Gonzalez', '147 Cedar St, Somewhere USA', '(890) 123-4567', 8)
(123467, 'Ashley Perez', '963 Pine St, Nowayville USA', '(901) 234-5678', 10),
(123468, 'Matthew Sanchez', '852 Oak St, Elsewhereland USA', '(012) 345-6789', 13
(123469, 'Jennifer Ramirez', '741 Elm St, Anytown USA', '(123) 456-7890', 9),
(123470, 'Christopher Torres', '369 Maple St, Elsewhereville USA', '(234) 567-890
(123471, 'Jessica Flores', '258 Pine St, Anywhereville USA', '(345) 678-9012', 7
(123472, 'Joshua Gomez', '963 Cedar St, Nowayland USA', '(456) 789-0123', 11),
(123473, 'Amanda Brooks', '852 Spruce St, Somewhereville USA', '(567) 890-1234',
(123474, 'James Bell', '741 Birch St, Anyplace USA', '(678) 901-2345', 13),
(123475, 'Danielle Ward', '369 Pine St, Nowhere USA', '(789) 012-3456', 6),
(123476, 'Andrew Cox', '258 Oak St, Elsewhere USA', '(890) 123-4567', 9);
(123477, 'Samantha Wood', '123 Elm St, Somewhereville USA', '(901) 234-5678', 10
(123478, 'Ryan Stewart', '456 Oak St, Nowhereville USA', '(012) 345-6789', 8),
(123479, 'Brittany Turner', '789 Pine St, Anyplace USA', '(123) 456-7890', 15),
(123480, 'Justin Parker', '321 Cedar St, Elsewhere USA', '(234) 567-8901', 7),
(123481, 'Taylor Cook', '654 Spruce St, Noway USA', '(345) 678-9012', 12),
(123482, 'Brandon Murphy', '987 Birch St, Anytown USA', '(456) 789-0123', 9),
(123483, 'Heather Rivera', '852 Maple St, Elsewhereville USA', '(567) 890-1234',
(123484, 'Eric Bailey', '741 Pine St, Nowhere USA', '(678) 901-2345', 11),
(123485, 'Kelsey Cooper', '369 Cedar St, Anyplaceville USA', '(789) 012-3456', 14
(123486, 'Nicholas Richardson', '258 Walnut St, Somewhere USA', '(890) 123-4567'
(123487, 'Kayla Cox', '147 Elm St, Nowayville USA', '(901) 234-5678', 10),
(123488, 'Hannah Howard', '963 Oak St, Elsewhereland USA', '(012) 345-6789', 13)
(123489, 'Zachary Flores', '852 Pine St, Anywhereville USA', '(123) 456-7890', 9
(123490, 'Megan Reed', '741 Cedar St, Nowayland USA', '(234) 567-8901', 12),
(123491, 'Austin Hill', '369 Spruce St, Somewhereville USA', '(345) 678-9012', 7
(123492, 'Lauren Ward', '258 Birch St, Anyplace USA', '(456) 789-0123', 11),
(123493, 'Cody Mitchell', '963 Maple St, Elsewhereville USA', '(567) 890-1234', 8
(123494, 'Stephanie Carter', '852 Pine St, Nowhere USA', '(678) 901-2345', 13),
(123495, 'Dylan Gray', '741 Cedar St, Anyplaceville USA', '(789) 012-3456', 6),
(123496, 'Taylor Brooks', '369 Elm St, Somewhere USA', '(890) 123-4567', 9),
(123497, 'Haley Coleman', '258 Oak St, Nowhereville USA', '(901) 234-5678', 10),
(123498, 'Joseph Rivera', '963 Pine St, Anytown USA', '(012) 345-6789', 8),
```

```
(123499, 'Alexis Long', '741 Cedar St, Elsewhere USA', '(123) 456-7890', 15),
(123500, 'Luke Diaz', '369 Spruce St, Noway USA', '(234) 567-8901', 7),
(123501, 'Kaitlyn Reed', '852 Cedar St, Anyplace USA', '(345) 678-9012', 12),
(123502, 'Gabriel Marshall', '741 Birch St, Elsewhereville USA', '(456) 789-0123
(123503, 'Madison Jackson', '963 Pine St, Nowhere USA', '(567) 890-1234', 6),
(123504, 'Aiden Bell', '852 Cedar St, Anyplaceville USA', '(678) 901-2345', 11),
(123505, 'Brianna Gonzalez', '741 Elm St, Somewhere USA', '(789) 012-3456', 14),
(123506, 'Logan Nelson', '369 Maple St, Nowhereville USA', '(890) 123-4567', 8),
(123507, 'Sydney Hill', '258 Pine St, Anywhereville USA', '(901) 234-5678', 10),
(123508, 'Austin Baker', '963 Cedar St, Nowayland USA', '(012) 345-6789', 13),
(123509, 'Maria Martinez', '852 Spruce St, Elsewhereland USA', '(123) 456-7890',
(123510, 'Tristan Thompson', '741 Oak St, Anytown USA', '(234) 567-8901', 12),
(123511, 'Emma Davis', '369 Birch St, Anyplace USA', '(345) 678-9012', 7),
(123512, 'Ethan White', '258 Cedar St, Somewhere USA', '(456) 789-0123', 11),
(123513, 'Olivia Clark', '963 Elm St, Elsewhereville USA', '(567) 890-1234', 8),
(123514, 'Avery Lewis', '852 Maple St, Nowhere USA', '(678) 901-2345', 13),
(123515, 'Peyton Roberts', '741 Pine St, Anyplaceville USA', '(789) 012-3456', 6
(123516, 'Natalie Adams', '369 Cedar St, Nowhereville USA', '(890) 123-4567', 9)
(123517, 'Cameron Wright', '258 Spruce St, Somewhereville USA', '(901) 234-5678'
(123518, 'Alexandra King', '963 Elm St, Noway USA', '(012) 345-6789', 8),
(123519, 'Julian Lee', '741 Oak St, Anytownville USA', '(123) 456-7890', 15),
(123520, 'Alyssa Perez', '369 Pine St, Elsewhere USA', '(234) 567-8901', 7),
(123521, 'Mason Young', '258 Birch St, Anyplace USA', '(345) 678-9012', 12),
(123522, 'Isabella Hall', '963 Cedar St, Nowayland USA', '(456) 789-0123', 9),
(123523, 'Jacob Scott', '741 Elm St, Somewhere USA', '(567) 890-1234', 6),
(123524, 'Grace Green', '369 Oak St, Elsewhereville USA', '(678) 901-2345', 11),
(123525, 'Elijah Hill', '258 Pine St, Anywhereville USA', '(789) 012-3456', 14),
(123526, 'Sophia Mitchell', '963 Cedar St, Nowhere USA', '(890) 123-4567', 8),
(123527, 'Jackson Young', '741 Spruce St, Anyplaceville USA', '(901) 234-5678', 1
(123528, 'Aubrey Harris', '369 Elm St, Nowhereville USA', '(012) 345-6789', 13),
(123529, 'Chloe King', '258 Cedar St, Somewhere USA', '(123) 456-7890', 9),
(123530, 'Landon Davis', '963 Maple St, Anyplace USA', '(234) 567-8901', 12),
(123531, 'Madeline Lee', '741 Pine St, Nowhereville USA', '(345) 678-9012', 7),
(123532, 'Gavin Martin', '369 Birch St, Elsewhere USA', '(456) 789-0123', 11),
(123533, 'Aaliyah White', '258 Oak St, Anywhere USA', '(567) 890-1234', 8),
(123534, 'Brooklyn Rodriguez', '963 Cedar St, Noway USA', '(678) 901-2345', 13),
(123535, 'Robert Scott', '741 Elm St, Anyplaceville USA', '(789) 012-3456', 6),
(123536, 'Leah Green', '369 Pine St, Somewhere USA', '(890) 123-4567', 9),
(123537, 'Wyatt Carter', '258 Cedar St, Nowhereville USA', '(901) 234-5678', 10)
```

```
(123538, 'Charlotte Adams', '963 Elm St, Elsewhere USA', '(012) 345-6789', 8),
(123539, 'Henry Lopez', '741 Pine St, Anyplace USA', '(123) 456-7890', 15),
(123540, 'Sofia Perez', '369 Cedar St, Noway USA', '(234) 567-8901', 7),
(123541, 'Zoe King', '258 Oak St, Anyplaceville USA', '(345) 678-9012', 12),
(123542, 'Lucas Thompson', '963 Cedar St, Somewhere USA', '(456) 789-0123', 9),
(123543, 'Kylie Davis', '741 Elm St, Nowayland USA', '(567) 890-1234', 6),
(123544, 'Gabriella Hill', '369 Pine St, Elsewhereland USA', '(678) 901-2345', 11
(123545, 'Dominic Nelson', '258 Cedar St, Anywhereville USA', '(789) 012-3456', 1
(123546, 'Gabrielle Hill', '963 Elm St, Nowhereville USA', '(890) 123-4567', 8),
(123547, 'Nathan Baker', '741 Oak St, Anytown USA', '(901) 234-5678', 10),
(123548, 'Addison Lee', '369 Birch St, Elsewhere USA', '(012) 345-6789', 13),
(123549, 'Hailey Carter', '258 Spruce St, Anyplaceville USA', '(123) 456-7890', 9
(123550, 'Jack Reed', '963 Elm St, Nowhereville USA', '(234) 567-8901', 12);

INSERT INTO Sales (SalesID, SalesName, SalesAddress, SalesRating, SalesPhoneNo, S
(2001, 'The Gourmet Kitchen', '123 Foodie Lane, Culinary City', 4.5, '(123) 456-7
(2002, 'Delicious Bites', '456 Flavor Avenue, Tasty Town', 4.0, '(456) 789-0123',
(2003, 'Savory Delights', '789 Chef Street, Flavorful Ville', 4.2, '(789) 012-345
(2004, 'Taste of Italy', '123 Pasta Lane, Italian City', 4.7, '(123) 456-7890', 5
(2005, 'Spice Fusion', '456 Spice Street, Flavor Town', 4.1, '(456) 789-0123', 38
(2006, 'Asian Delight', '789 Wok Avenue, Asian Cuisine Ville', 4.4, '(789) 012-34
(2007, 'Mexican Fiesta', '123 Taco Lane, Salsa City', 4.3, '(123) 456-7890', 4800
(2008, 'Burger Haven', '456 Patty Street, Burger Town', 4.2, '(456) 789-0123', 41
(2009, 'Sushi Paradise', '789 Roll Avenue, Sashimi Ville', 4.6, '(789) 012-3456',
(2010, 'Mediterranean Delicacies', '123 Olive Lane, Med City', 4.4, '(123) 456-78
(2011, 'Seafood Sensation', '456 Ocean Avenue, Fish Town', 4.5, '(456) 789-0123',
(2012, 'Veggie Delight', '789 Greens Street, Veggie Ville', 4.0, '(789) 012-3456
(2013, 'BBQ Heaven', '123 Grill Lane, BBQ City', 4.8, '(123) 456-7890', 7000),
(2014, 'Sweet Treats Bakery', '456 Sugar Street, Dessert Town', 4.2, '(456) 789-0
(2015, 'Coffee Craze', '789 Bean Avenue, Caffeine Ville', 4.3, '(789) 012-3456',
(2016, 'French Flavors', '123 Croissant Lane, Paris City', 4.6, '(123) 456-7890',
(2017, 'Steakhouse Supreme', '456 Beef Street, Meat Town', 4.7, '(456) 789-0123',
(2018, 'Greek Taverna', '789 Gyro Avenue, Opa Ville', 4.5, '(789) 012-3456', 5400
(2019, 'Healthy Haven', '123 Wellness Lane, Fit City', 4.0, '(123) 456-7890', 400
(2020, 'Pizzeria Paradiso', '456 Slice Street, Pizza Town', 4.4, '(456) 789-0123
(2021, 'Tea Time', '789 Leaf Avenue, Tea Ville', 4.2, '(789) 012-3456', 4500),
(2022, 'Ramen House', '123 Noodle Lane, Ramen City', 4.6, '(123) 456-7890', 5900
(2023, 'Juice Junction', '456 Fruit Street, Juice Town', 4.1, '(456) 789-0123', 4
(2024, 'Vegetarian Village', '789 Garden Avenue, Herb Ville', 4.3, '(789) 012-345
```

```
(2025, 'Dim Sum Palace', '123 Dumpling Lane, Dim City', 4.5, '(123) 456-7890', 56
(2026, 'Waffle Wonderland', '456 Syrup Street, Waffle Town', 4.4, '(456) 789-0123
(2027, 'Bagel Cafe', '789 Dough Avenue, Bagel Ville', 4.0, '(789) 012-3456', 3700
(2028, 'Creperie Charm', '123 Crepe Lane, Crepe City', 4.2, '(123) 456-7890', 430
(2029, 'Taco Truck', '456 Taco Street, Taco Town', 4.3, '(456) 789-0123', 4900),
(2030, 'Hot Pot Haven', '789 Broth Avenue, Hot Pot Ville', 4.7, '(789) 012-3456',

INSERT INTO Sales (SalesName, SalesAddress, SalesRating, SalesPhoneNo, SalesCount
(301201,'Delicious Bites', '456 Yummy Street, Food Town', 4.2, '(456) 789-0123',
(301202,'Savory Spices', '789 Flavor Avenue, Tasty Town', 4.7, '(789) 012-3456',
(301203,'Tasty Treats', '321 Savory Road, Flavor City', 4.4, '(321) 654-9870', 45
(301204,'Yum Yum Delights', '987 Tasty Lane, Foodie City', 4.6, '(987) 654-3210',
(301205,'Flavor Fusion', '654 Delish Drive, Yum Town', 4.3, '(654) 321-0987', 410
(301206,'Taste of Heaven', '210 Flavorful Boulevard, Yummy City', 4.8, '(210) 987
(301207,'Sizzling Senses', '543 Delicious Street, Flavor Town', 4.1, '(543) 210-9
(301208'Zesty Zing', '876 Savory Lane, Taste City', 4.9, '(876) 543-2109', 7400)
(301209,'Flavorful Feast', '109 Tasty Avenue, Delish Town', 4.2, '(109) 876-5432
(301210,'Yummy Yields', '234 Tasty Drive, Flavor City', 4.7, '(234) 567-8901', 62
(301211,'Delightful Dining', '567 Delicious Road, Yum Town', 4.4, '(567) 890-1234
(301212,'Savory Sips', '890 Flavorful Boulevard, Tasty City', 4.6, '(890) 123-456
(301213'Tempting Tastes', '432 Savory Street, Flavor Town', 4.3, '(432) 109-8765
(301214,'Tasty Temptations', '765 Yummy Lane, Delish City', 4.8, '(765) 432-1098
(301215,'Flavor Fiesta', '321 Tasty Avenue, Savory Town', 4.1, '(321) 098-7654',
(301216,'Yum Factor', '654 Delicious Drive, Tasty City', 4.9, '(654) 321-0987', 7
(301217,'Savory Sensations', '987 Flavorful Road, Yum Town', 4.2, '(987) 654-3210
(301218,'Taste Bud Haven', '210 Tasty Boulevard, Flavor City', 4.7, '(210) 987-65
(301219,'Delicious Discoveries', '543 Flavor Street, Yummy Town', 4.4, '(543) 210
(301220,'Yum Haven', '876 Tasty Lane, Delish City', 4.6, '(876) 543-2109', 5300)
(301221,'Tasty Trails', '109 Delicious Avenue, Flavor Town', 4.3, '(109) 876-5432
(301222,'Savory Supremacy', '234 Flavor Drive, Tasty City', 4.8, '(234) 567-8901
(301223,'Flavor Frenzy', '567 Tasty Road, Savory Town', 4.1, '(567) 890-1234', 32
(301224,'Yum Sensations', '890 Delicious Boulevard, Flavor City', 4.9, '(890) 123
(301225,'Taste Haven', '432 Tasty Street, Yum Town', 4.2, '(432) 109-8765', 3800)
(301226,'Delish Delights', '765 Flavorful Lane, Tasty City', 4.7, '(765) 432-1098
(301227'Savory Sizzle', '098 Delicious Avenue, Yummy Town', 4.5, '(098) 765-4321
(301228,'Tasty Times', '876 Tasty Drive, Flavor City', 4.4, '(876) 543-2109', 450
(301229'Yum Fusion', '210 Delicious Road, Tasty Town', 4.6, '(210) 987-6543', 530
(301230,'Flavor Haven', '543 Flavorful Boulevard, Yum City', 4.3, '(543) 210-9876
```

```
INSERT INTO Food (FoodID, SalesID, FoodName, FoodPrice, FoodRating) VALUES
(101, 301201, 'Chicken Parmesan', 10.00, 4.5),
(102, 301201, 'Spaghetti Bolognese', 12.00, 4.2),
(103, 301201, 'Pizza Margherita', 11.00, 4.8),
(104, 301202, 'Chicken Tikka Masala', 10.50, 4.7),
(105, 301202, 'Butter Chicken', 12.50, 4.4),
(106, 301202, 'Lamb Korma', 11.50, 4.6),
(107, 301203, 'Fish and Chips', 10.20, 4.3),
(108, 301203, 'Bangers and Mash', 12.20, 4.5),
(109, 301203, 'Toad in the Hole', 11.20, 4.7),
(110, 301204, 'Beef Wellington', 10.10, 4.6),
(111, 301204, 'Roast Chicken', 12.10, 4.8),
(112, 301204, 'Yorkshire Pudding', 11.10, 4.5),
(113, 301205, 'Lasagna', 10.05, 4.4),
(114, 301205, 'Pasta Carbonara', 12.05, 4.6),
(115, 301205, 'Fettuccine Alfredo', 11.05, 4.7),
(116, 301206, 'Sushi', 10.03, 4.5),
(117, 301206, 'Sashimi', 12.03, 4.7),
(118, 301206, 'Tempura', 11.03, 4.6),
(119, 301207, 'Sweet and Sour Chicken', 10.02, 4.2),
(120, 301207, 'Kung Pao Chicken', 12.02, 4.4),
(121, 301207, 'Chicken Satay', 11.02, 4.8),
(122, 301208, 'Chicken Tandoori', 10.01, 4.3),
(123, 301208, 'Chicken Curry', 12.01, 4.5),
(124, 301208, 'Chicken Vindaloo', 11.01, 4.7),
(125, 301209, 'Chicken Wings', 10.04, 4.8),
(126, 301209, 'Buffalo Wings', 12.04, 4.6),
(127, 301209, 'Teriyaki Chicken', 11.04, 4.5),
(128, 301210, 'Chicken Rice', 10.06, 4.7),
(129, 301210, 'Nasi Lemak', 12.06, 4.5),
(130, 301210, 'Nasi Goreng', 11.06, 4.4),
(131, 301211, 'Chicken Noodle Soup', 10.07, 4.6),
(132, 301211, 'Chicken Pot Pie', 12.07, 4.8),
(133, 301211, 'Chicken Cordon Bleu', 11.07, 4.5),
(134, 301212, 'Chicken Caesar Salad', 10.08, 4.5),
(135, 301212, 'Chicken Salad', 12.08, 4.7),
(136, 301212, 'Chicken Wrap', 11.08, 4.6),
(137, 301213, 'Chicken Burger', 10.09, 4.4),
(138, 301213, 'Chicken Sandwich', 12.09, 4.6),
```

```
(139, 301213, 'Chicken Quesadilla', 11.09, 4.8),
(140, 301214, 'Chicken Fried Rice', 10.10, 4.3),
(141, 301214, 'Chicken Fried Steak', 12.10, 4.5),
(142, 301214, 'Chicken Fried Chicken', 11.10, 4.7),
(143, 301215, 'Chicken and Dumplings', 10.11, 4.2),
(144, 301215, 'Chicken and Rice', 12.11, 4.4),
(145, 301215, 'Chicken and Mushroom Soup', 11.11, 4.6),
(146, 301216, 'Chicken and Broccoli Casserole', 10.12, 4.1),
(147, 301216, 'Chicken and Rice Bake', 12.12, 4.3),
(148, 301216, 'Chicken and Asparagus', 11.12, 4.5),
(149, 301217, 'Chicken and Tomato Soup', 10.13, 4.0),
(150, 301217, 'Chicken and Noodle Soup', 12.13, 4.2),
(151, 301217, 'Chicken and Vegetable Soup', 11.13, 4.4),
(152, 301218, 'Chicken and Potato Soup', 10.14, 4.1),
(153, 301218, 'Chicken and Pasta Soup', 12.14, 4.3),
(154, 301218, 'Chicken and Gravy', 11.14, 4.5),
(155, 301219, 'Chicken and Dumpling Soup', 10.15, 4.0),
(156, 301219, 'Chicken and Rice Soup', 12.15, 4.2),
(157, 301219, 'Chicken and Mushroom Soup', 11.15, 4.4),
(158, 301220, 'Chicken and Broccoli Soup', 10.16, 4.1),
(159, 301220, 'Chicken and Rice Soup', 12.16, 4.3),
(160, 301220, 'Chicken and Asparagus Soup', 11.16, 4.5);
(161, 301221, 'Chicken and Tomato Soup', 10.17, 4.0),
(162, 301221, 'Chicken and Noodle Soup', 12.17, 4.2),
(163, 301221, 'Chicken and Vegetable Soup', 11.17, 4.4),
(164, 301222, 'Chicken and Potato Soup', 10.18, 4.1),
(165, 301222, 'Chicken and Pasta Soup', 12.18, 4.3),
(166, 301222, 'Chicken and Gravy', 11.18, 4.5),
(167, 301223, 'Chicken and Dumpling Soup', 10.19, 4.0),
(168, 301223, 'Chicken and Rice Soup', 12.19, 4.2),
(169, 301223, 'Chicken and Mushroom Soup', 11.19, 4.4),
(170, 301224, 'Chicken and Broccoli Soup', 10.20, 4.1),
(171, 301224, 'Chicken and Rice Soup', 12.20, 4.3),
(172, 301224, 'Chicken and Asparagus Soup', 11.20, 4.5),
(173, 301225, 'Chicken and Tomato Soup', 10.21, 4.0),
(174, 301225, 'Chicken and Noodle Soup', 12.21, 4.2),
(175, 301225, 'Chicken and Vegetable Soup', 11.21, 4.4),
(176, 301226, 'Chicken and Potato Soup', 10.22, 4.1),
(177, 301226, 'Chicken and Pasta Soup', 12.22, 4.3),
```

```
(178, 301226, 'Chicken and Gravy', 11.22, 4.5),
(179, 301227, 'Chicken and Dumpling Soup', 10.23, 4.0),
(180, 301227, 'Chicken and Rice Soup', 12.23, 4.2),
(181, 301227, 'Chicken and Mushroom Soup', 11.23, 4.4),
(182, 301228, 'Chicken and Broccoli Soup', 10.24, 4.1),
(183, 301228, 'Chicken and Rice Soup', 12.24, 4.3),
(184, 301228, 'Chicken and Asparagus Soup', 11.24, 4.5),
(185, 301229, 'Chicken and Tomato Soup', 10.25, 4.0),
(186, 301229, 'Chicken and Noodle Soup', 12.25, 4.2),
(187, 301229, 'Chicken and Vegetable Soup', 11.25, 4.4),
(188, 301230, 'Chicken and Potato Soup', 10.26, 4.1),
(189, 301230, 'Chicken and Pasta Soup', 12.26, 4.3),
(190, 301230, 'Chicken and Gravy', 11.26, 4.5);

INSERT INTO `Order` (OrderID, OrderTime, TotalAmount, CustomerID, SalesID) VALUES
(2011, '2022-01-11 20:00', 70.00, 123467, 301211),
(2012, '2022-01-12 21:00', 75.00, 123468, 301211),
(2013, '2022-01-13 22:00', 80.00, 123469, 301211),
(2014, '2022-01-14 23:00', 85.00, 123470, 301214),
(2015, '2022-01-15 00:00', 90.00, 123471, 301215),
(2016, '2022-01-16 01:00', 95.00, 123472, 301216),
(2017, '2022-01-17 02:00', 100.00, 123473, 301217),
(2018, '2022-01-18 03:00', 105.00, 123474, 301212),
(2019, '2022-01-19 04:00', 110.00, 123475, 301219),
(2020, '2022-01-20 05:00', 115.00, 123476, 301220),
(2021, '2022-01-21 06:00', 120.00, 123477, 301221),
(2022, '2022-01-22 07:00', 125.00, 123478, 301222),
(2023, '2022-01-23 08:00', 130.00, 123479, 301223),
(2024, '2022-01-24 09:00', 135.00, 123480, 301222),
(2025, '2022-01-25 10:00', 140.00, 123481, 301222),
(2026, '2022-01-26 11:00', 145.00, 123482, 301226),
(2027, '2022-01-27 12:00', 150.00, 123483, 301227),
(2028, '2022-01-28 13:00', 155.00, 123484, 301227),
(2029, '2022-01-29 14:00', 160.00, 123485, 301229),
(2030, '2022-01-30 15:00', 165.00, 123486, 301230),
(2031, '2022-01-31 16:00', 170.00, 123487, 301231),
(2032, '2022-02-01 17:00', 175.00, 123488, 301232),
(2033, '2022-02-02 18:00', 180.00, 123489, 301233),
(2034, '2022-02-03 19:00', 185.00, 123490, 301234),
```

```
(2035, '2022-02-04 20:00', 190.00, 123491, 301235),
(2036, '2022-02-05 21:00', 195.00, 123492, 301236),
(2037, '2022-02-06 22:00', 200.00, 123493, 301237),
(2038, '2022-02-07 23:00', 205.00, 123494, 301238),
(2039, '2022-02-08 00:00', 210.00, 123495, 301222);

INSERT INTO ORDERLINE VALUES
(2011,101,301211,2),
(2012,101,301212,3),
(2013,101301213,3),
(2014,110,301214,2),
(2015,113,301215,4),
(2016,119,301216,5),
(2017,122,301217,7),
(2018,106,301218,3),
(2019,128,301219,4),
(2020,131,301220,5),
(2021,134,302021,5),
(2022,137,302022,3),
(2023,140,301223,4),
(2024,137,301224,4),
(2025,137,301225,2),
(2026,149,301226,3),
(2027,152,301227,4),
(2028,152,301228,5),
(2029,158,301229,2),
(2030,161,301230,6),
(2031,164,301231,3),
(2032,167,301232,5),
(2033,170,301233,6),
(2034,173,301234,2),
(2035,177,301235,4),
(2036,180,301236,6),
(2037,183,301237,5),
(2038,186,301238,7),
(2039,189,301239,2);


INSERT INTO DELIVERY VALUES
```

```
(1001,'2023-04-12 09:00:00', '2023-04-12 10:20:00', 4.7, 5.2),
(1002,'2023-04-13 11:30:00', '2023-04-13 13:00:00', 4.9, 8.6),
(1003,'2023-04-14 14:00:00', '2023-04-14 15:45:00', 4.6, 12.1),
(1004,'2023-04-15 16:15:00', '2023-04-15 17:40:00', 4.8, 7.4),
(1005'2023-04-16 18:45:00', '2023-04-16 20:15:00', 4.5, 3.8),
(1006,'2023-04-17 21:00:00', '2023-04-17 22:30:00', 5.0, 15.5);
(1007'2023-04-18 09:30:00', '2023-04-18 10:50:00', 4.8, 6.2),
(1008'2023-04-19 12:00:00', '2023-04-19 13:30:00', 4.5, 9.8),
(1009,'2023-04-20 14:30:00', '2023-04-20 16:15:00', 4.9, 11.3),
(1010,'2023-04-21 16:45:00', '2023-04-21 18:10:00', 4.7, 8.2),
(1011,'2023-04-22 18:30:00', '2023-04-22 20:00:00', 4.6, 4.5),
(1012,'2023-04-23 20:45:00', '2023-04-23 22:15:00', 4.8, 6.9),
(1013'2023-04-24 09:00:00', '2023-04-24 10:20:00', 4.5, 5.7),
(1014,'2023-04-25 11:30:00', '2023-04-25 13:00:00', 4.7, 7.1),
(1015,'2023-04-26 14:00:00', '2023-04-26 15:45:00', 4.9, 10.4),
(1016,'2023-04-27 16:15:00', '2023-04-27 17:40:00', 4.6, 9.2),
(1017,'2023-04-28 18:45:00', '2023-04-28 20:15:00', 4.8, 6.6),
(1018,'2023-04-29 21:00:00', '2023-04-29 22:30:00', 4.7, 12.8),
(1019,'2023-04-30 09:30:00', '2023-04-30 10:50:00', 4.9, 8.3),
(1020,'2023-05-01 12:00:00', '2023-05-01 13:30:00', 4.6, 5.9),
(1021,'2023-05-02 14:30:00', '2023-05-02 16:15:00', 4.8, 9.6),
(1022,'2023-05-03 16:45:00', '2023-05-03 18:10:00', 4.7, 11.2),
(1023,'2023-05-04 18:30:00', '2023-05-04 20:00:00', 4.5, 6.4),
(1024,'2023-05-05 20:45:00', '2023-05-05 22:15:00', 4.9, 8.7),
(1025,'2023-05-06 09:00:00', '2023-05-06 10:20:00', 4.6, 4.8),
(1026,'2023-05-07 11:30:00', '2023-05-07 13:00:00', 4.8, 7.9),
(1027,'2023-05-08 14:00:00', '2023-05-08 15:45:00', 4.7, 10.1),
(1028,'2023-05-09 16:15:00', '2023-05-09 17:40:00', 4.9, 11.8);

INSERT INTO Delivery (DelivererID, DelivererName, DelivererRating, DelivererCount
VALUES
(100001, 'John Doe', 4.7, 10),
(100002, 'Jane Smith', 4.9, 8),
(100003, 'Michael Johnson', 4.6, 12),
(100004, 'Sarah Lee', 4.8, 15),
(100005, 'David Kim', 4.5, 7),
(100006, 'Emily Chen', 5.0, 9);
```